

# COS 214 Practical 5

## CampusGuard: Emergency Response Coordination

**Date Released:** 15 September 2026  
**Date Due:** 29 September 2026  
**Team Size:** 3 students  
**Language:** C++11  
**Core topics:** Command, Mediator, Adapter, Facade, two additional GoF patterns, UML, GDB, Valgrind, Docker, Git/GitHub

### 1 Introduction

Practical 5 is deliberately closer to the semester project than the earlier practicals. You are given a single scenario and a set of required design patterns. Your task is to decide how the system should be designed, where each pattern belongs, how the patterns collaborate, and how the resulting C++ implementation remains coherent.

You are **not** given a class structure. You are given the problem and the patterns that must appear meaningfully in your solution. Deciding what objects are needed, where responsibilities belong, how collaborations meet, what is owned, and how the system should behave is part of the practical.

Your solution must make meaningful use of the following Gang of Four patterns:

- Command;
- Mediator;
- Adapter;
- Facade; and
- **two additional GoF patterns of your team's choice.**

The final solution must behave like one application, not six unrelated pattern demonstrations. The purpose of the practical is to prepare you for the project, where you will need to recognise design problems and choose suitable patterns rather than simply reproduce isolated examples.

### 2 Learning Outcomes

After completing this practical you should be able to:

- recognise multiple design-pattern opportunities inside one realistic scenario;
- combine behavioural and structural patterns without creating unnecessary coupling;
- use Command to represent requests as objects and separate request creation from execution;
- use Mediator to coordinate collaborating objects without direct many-to-many dependencies;
- use Adapter to integrate an existing or incompatible interface into a design;
- use Facade to provide a useful higher-level interface to a more complex subsystem;
- select two additional GoF patterns and justify why they improve the design;
- produce UML that accurately reflects the implemented system; and
- explain design decisions, ownership, trade-offs and pattern interactions during a live demonstration.

### 3 Non-negotiable Rules

1. Complete this practical in the **allocated team of three**. All three members must understand the complete submission.
2. Compile using `-std=c++11`. The project must be built using a Makefile; the marker will build it by running `make`.
3. Every polymorphic base class must have a virtual destructor.
4. Every owning relationship must have a clear and defensible destruction policy.
5. Command, Mediator, Adapter and Facade must all appear meaningfully in the final design.
6. Your team must add **exactly two additional GoF patterns** of its choice. They must solve genuine design problems in the scenario.
7. A single class may participate in more than one pattern only where the design genuinely justifies it.
8. The required patterns may not be replaced by giant switch statements, large centralised if/else chains, type checks, or manual client code that knows every concrete class.
9. Your UML and implementation must agree.
10. The implementation must form one coherent application. A menu that merely runs six isolated pattern examples will not satisfy the practical.
11. The complete application must build and run inside Docker. The assessed demonstration must be launched with `docker compose up -build`; running a locally compiled executable instead will not count as the demonstration.

**GITHUB IS COMPULSORY. A repository created or populated only at the end does not count as meaningful GitHub use. The repository history must show development over time and meaningful contributions from all three team members.**

### 4 Scenario and System Requirements

Your team will design and implement **CampusGuard**, an emergency-response coordination platform for a large university campus. The campus has security teams, medical responders, facilities staff, access-control systems and communication services. Some of these systems are modern, while others are legacy or externally managed and expose incompatible interfaces. During an incident, actions must be coordinated quickly without every component depending directly on every other component.

The platform must support staff from the moment an incident is reported until it is resolved. The scenario includes the following operational areas:

- registering incidents and changing their operational status;
- dispatching campus security, medical responders or facilities staff;
- building access actions such as locking, unlocking or restricting selected areas;
- operator actions such as dispatching a response unit, issuing an evacuation instruction, activating an emergency alert or cancelling an action;
- coordination between response components when the condition of an incident changes;
- integration with at least one existing external or legacy service whose interface does not match the interface CampusGuard wants to use; and
- high-level emergency workflows that require several subsystem operations to happen in a controlled sequence.

You are not given a class structure. You must decide what objects are needed, what responsibilities belong where, which objects own other objects, and how the required patterns solve the problems in the

scenario.

## 5 Required Design Pattern Behaviour

Your implementation must use each required pattern for its intended design purpose. The descriptions below specify the behaviour that must be visible in the final system; they do not prescribe your class names or exact structure.

### 5.1 Command

Requests or operator actions must be represented as objects. Your design must include at least three meaningful concrete commands and appropriate invoker and receiver roles. The commands must do more than print their own names; they must trigger real domain behaviour in CampusGuard. Suitable actions could include dispatching a unit, securing an area, issuing an alert or cancelling a previous instruction.

### 5.2 Mediator

A group of response components must collaborate through a mediator rather than maintaining direct many-to-many knowledge of one another. At least one meaningful change or action in one colleague must result in the mediator coordinating behaviour involving other colleagues.

### 5.3 Adapter

CampusGuard must integrate at least one external or legacy service whose interface is incompatible with the interface your application wants to use. The adapter must translate between these interfaces. Simply wrapping a class without changing the expected interface is not sufficient.

### 5.4 Facade

Provide a higher-level entry point for a realistic workflow that would otherwise require the client to interact with several subsystem services. At least one facade operation must coordinate three or more subsystem operations. The underlying subsystem operations must still remain independently usable where appropriate.

### 5.5 Two Additional Patterns

Select two additional GoF patterns that improve your design. You must be able to explain:

- what design problem each pattern solves;
- why that pattern is a better choice than a simpler alternative; and
- how it participates in the integrated CampusGuard workflow.

## 6 Functional Requirements

Your running system must demonstrate all of the following:

- at least two incidents or operational contexts with different runtime data;
- at least four collaborating response components or services;
- at least three concrete commands that operate on meaningful receivers;
- one sequence in which a command triggers behaviour that causes other components to coordinate through the mediator;
- one external or legacy service accessed through an adapter;
- one facade operation that performs a realistic multi-step workflow using at least three subsystem services;
- both additional chosen patterns participating in normal application behaviour;
- at least one failure or invalid-operation case handled sensibly rather than silently ignored; and
- runtime output that makes the important collaborations visible enough for a tutor to follow during the demonstration.

## 7 Tasks

### Task 1: Analyse the Scenario and Design the System

Before implementation, identify the main design problems in CampusGuard and produce a complete design. Your submitted design must include:

- a concise statement of the incident-response responsibilities your system will support;
- a complete UML class diagram drawn from scratch;
- identification of the GoF participants for Command, Mediator, Adapter and Facade in your own design;
- the two additional GoF patterns selected by your team and a short justification for each;
- important associations, inheritance relationships, multiplicities and ownership decisions; and
- a short explanation of where the six patterns interact rather than operate independently.

Do not design toward a fixed number of classes. The design should be as small as it can reasonably be while still supporting the required behaviour.

### Task 2: Implement the Integrated Model

Implement the design in C++11. The code must follow the UML and use meaningful abstractions. Do not name classes only after pattern roles unless that name is also natural in the domain. The goal is a CampusGuard application that happens to use good patterns, not a collection of textbook skeletons.

### Task 3: Build Two End-to-End Scenarios

Create two non-trivial runtime stories. Across the two stories, the tutor must be able to observe all six patterns collaborating. A suitable story may, for example, begin with an incident being reported, dispatch one or more response teams, coordinate a building-access change, notify affected components, and interact with a legacy or external service. Your exact stories are your design decision.

Each scenario must include more than one pattern. At least one scenario must involve **four or more of the six patterns in the same execution flow**.

### Task 4: UML Diagram Portfolio

Submit a diagram portfolio that explains the design and important workflows in your actual implementation. Include:

- one complete UML class diagram for the final system;
- one UML sequence diagram showing a substantial runtime interaction that includes Command plus at least one other required pattern;
- one second UML sequence diagram showing a different workflow that includes the Facade or Adapter; and
- one additional behavioural diagram of your choice (activity, state or sequence) that best explains one of your two chosen patterns.

The diagrams are part of the design. Important messages, operations and relationships shown in the diagrams must be traceable to the final C++ implementation.

### Task 5: Engineering Quality

Your submission must show that the system was engineered rather than assembled only for the demonstration. Include:

- a Makefile that builds the complete application;

- a working Dockerfile and `docker-compose.yml` that build and run the complete application;
- a README with the exact Docker build/run instructions, including `docker compose up -build`;
- clean polymorphic destruction and a clear ownership policy;
- Valgrind evidence for the final executable;
- a short GDB investigation of a real execution bug; and
- meaningful Git/GitHub history from all three team members.

## Task 6: Integration and Demonstration

Prepare a coherent demonstration of approximately five to seven minutes. Do not demonstrate one pattern at a time. Tell the story of the system and let the patterns appear naturally as the workflow executes.

**The demonstration must be run through Docker.** At the start of the demo, your team must launch the complete application using `docker compose up -build`. The tutor must be able to see Docker build the image and start the application/container. Do not pre-build and then demonstrate only a local executable. Your Docker setup must therefore reproduce the same system that you are presenting and must work from the submitted repository.

During the demonstration, a tutor may ask any team member to:

- identify the GoF participants for any of the six patterns;
- trace a command from invoker to receiver;
- explain why a particular interaction belongs in the mediator rather than in a colleague;
- show what interface is being adapted and why the adapter is needed;
- show which complexity the facade hides and which subsystem operations remain independently accessible;
- justify either of the two additional patterns;
- explain an ownership or lifetime decision;
- trace a UML message to the corresponding C++ method;
- rebuild and launch the application using `docker compose up -build`;
- run or discuss Valgrind/GDB evidence inside the Docker environment; or
- explain how the team divided and integrated the work.

All three team members are responsible for understanding the complete submission.

## 8 Submission

Submit one source archive and one PDF. The GitHub repository must remain accessible to the teaching team until marking is complete.

The PDF should contain, in a sensible order:

- team member details;
- short system description;
- final UML class diagram;
- GoF participant mapping for all six patterns;
- justification for the two team-selected patterns;
- the three behavioural diagrams;

- concise design and ownership rationale;
- GDB investigation and Valgrind evidence;
- Dockerfile, `docker-compose.yml`, and the Docker run instructions used for the demonstration;
- GitHub repository link and short collaboration reflection; and
- a short contribution statement for each team member.

## 9 Final Advice

Begin with the scenario, not with six pattern skeletons. Decide what an emergency-response system needs to do, where coupling or incompatible interfaces create problems, and where clients need a simpler entry point. Apply patterns only where they genuinely improve the design. The strongest submissions will have UML, implementation and runtime behaviour that tell one coherent story.